pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Decision Trees

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

A model that asks questions

Ask

a question of one column

Split

the accounts in two

Stop

the hard part: today's lesson

Allowed names

model probability of default in this file

flagged at threshold t

the model's score

Never called

a credit decision or an approval

a lending policy or a lending rule

a risk grade for a real person

Real UCI data: 30,000 accounts, Taiwan, 2005. No person column is a feature. Test rows stay closed.

How a decision tree works

Start	All the training accounts in one group: the root
Ask	Try every column and cut point; keep the question that leaves the two groups most alike in outcome
Repeat	Send each account left or right, then do the same on each side, until a rule or pure groups stop it
The leaf	An account's path ends in a leaf; the output is the share of defaults among the training accounts there
You control	Depth, leaf size, class weights, and the rest of the settings

A tree as questions

Word	Meaning
Node	A group of training accounts
Root	The first node: every account
Split	Is this value at most t? Yes left
Leaf	A node with no question
Depth	Questions from root to deepest leaf

```
|--- PAY_0 <= 1.50
|   |--- weights: [13455.00, 2660.00] class: 0
|--- PAY_0 >  1.50
|   |--- weights: [563.00, 1322.00] class: 1
```

18,000 training accounts. Left: codes -2, -1, 0, 1 (two undocumented). Right: a delay of two months or more.

PAY_0 <= 1.5: how this file splits, in its training rows. Not a rule for lending.

How Gini impurity works

The need	A way to say which question is best
Gini	How mixed a group is: 0 if all one outcome, at most 0.5 for a half-and-half group
Weighted Gini	The Gini of each side, weighted by its share of the accounts, added up
The score	The decrease from the parent's Gini; the tree keeps the largest
What comes out	A score for every candidate question, and one winner for each node

Gini impurity, and a split by hand

```
Gini = 1 - p^2 - (1 - p)^2     p = share of defaults;  0 pure, 0.5 half and half
```

Group	Accounts	Defaults	Share	Gini
all training rows	18,000	3,982	0.2212	0.3446
left: PAY_0 <= 1.5	16,115	2,660	0.1651	0.2756
right: PAY_0 > 1.5	1,885	1,322	0.7013	0.4189

0.2906

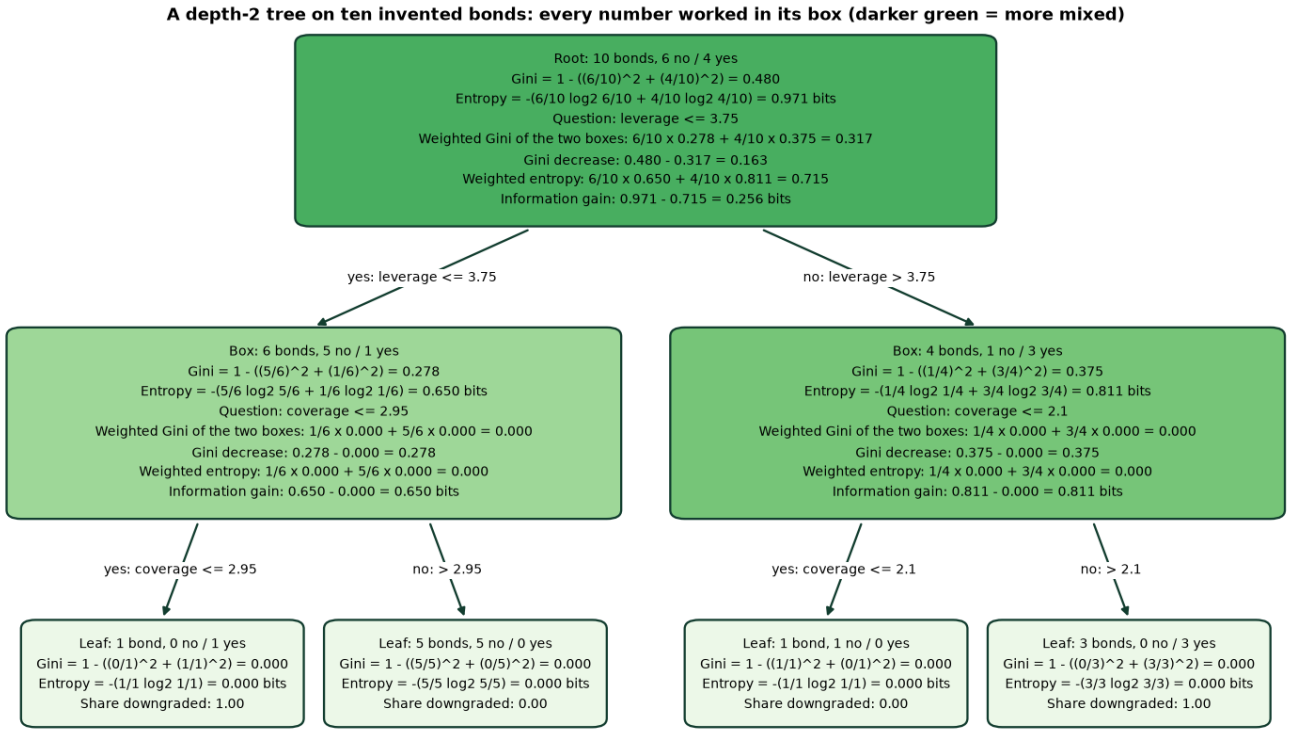
weighted Gini of the two sides

0.0539

decrease: the largest of any question

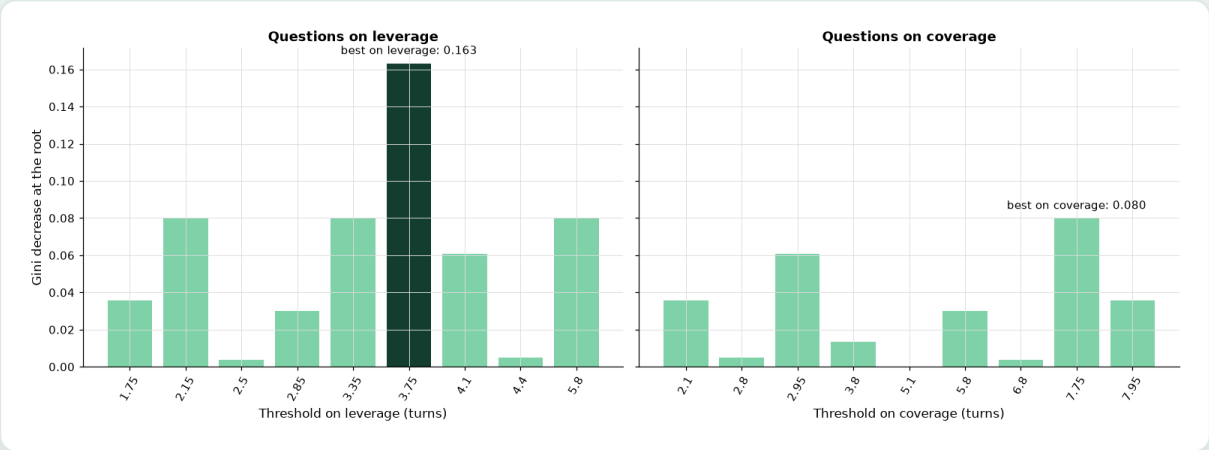
Four counts by COUNTIFS. Excel does it too.

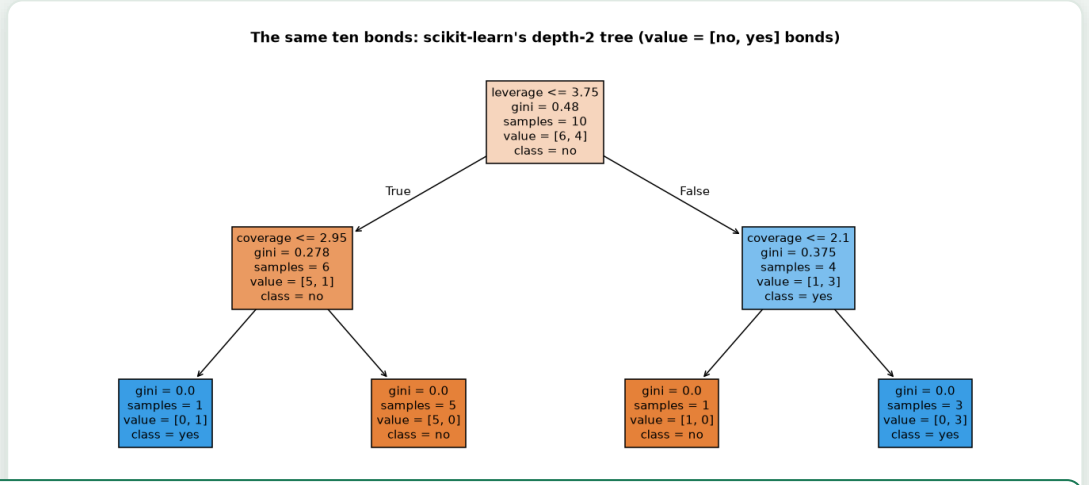
Gini in boxes: ten invented bonds



Root: Gini 0.480, question leverage <= 3.75, weighted Gini 0.317, decrease 0.163. Pure boxes score 0.

Which question wins, and scikit-learn agrees



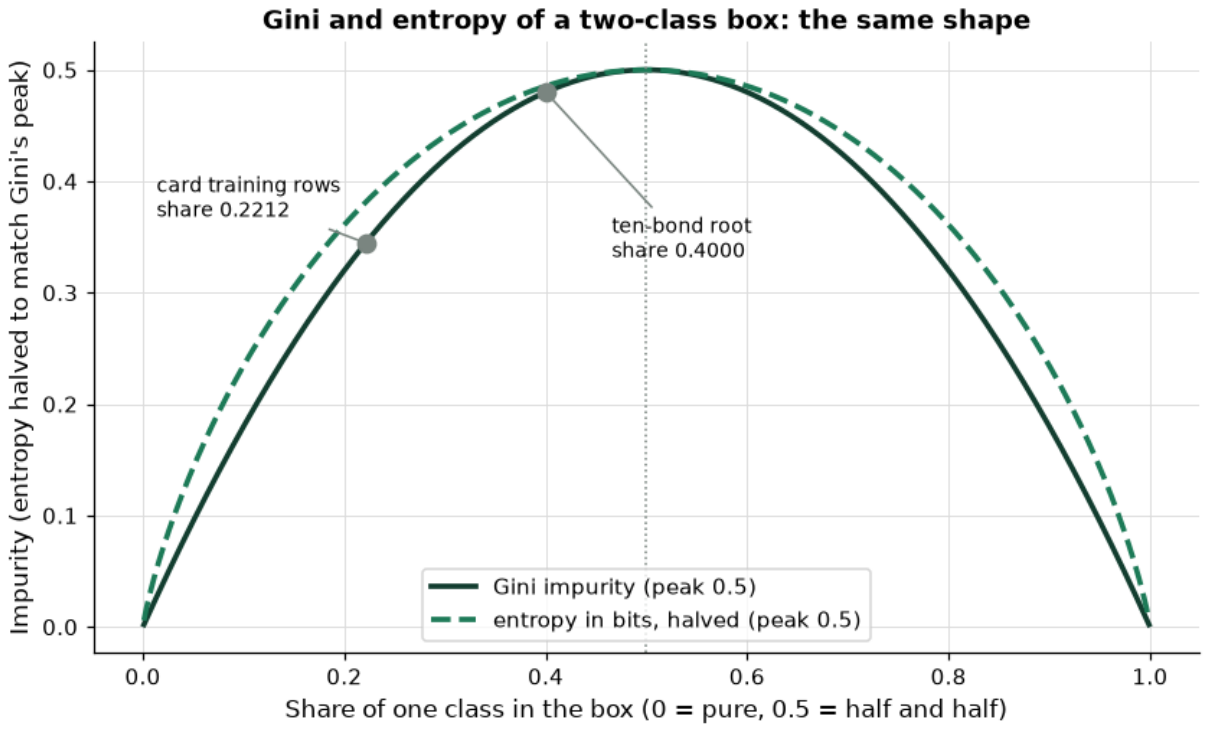


Winner: leverage <= 3.75, decrease 0.1633. Best on the other column: coverage <= 7.75, 0.0800. 7 of 7 boxes match scikit-learn's tree.

How entropy and information gain work

Entropy	A second way to score how mixed a group is, in bits
The range	0 for a pure group, 1 bit for a half-and-half group
Information gain	The parent's entropy minus the weighted entropy of the two sides
The choice	The tree keeps the largest gain, as it keeps Gini's largest decrease
You control	criterion="entropy" against the default "gini"; on two classes they usually rank alike

Entropy and information gain



Entropy, in bits

-(p log2 p + (1 - p) log2 (1 - p)): 0 when pure, 1 bit at fifty fifty

Information gain at the root

0.971 - 0.715 = 0.256 bits: the parent's entropy minus the weighted entropy of the two boxes

The two usually pick the same split

Gini, entropy, and log_loss (the same as entropy) chose the same root question on the card file and on the ten bonds.

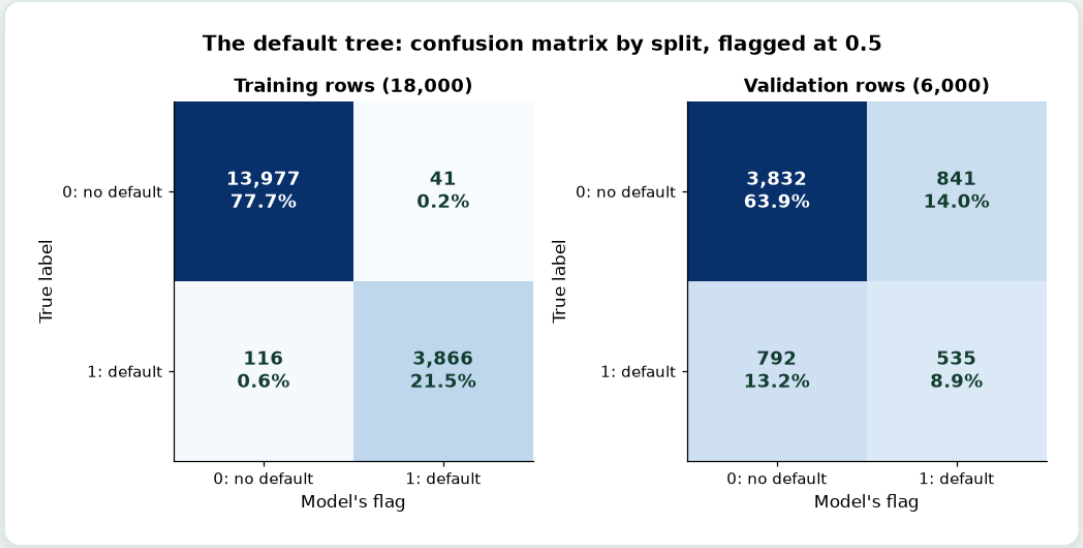
The default tree overfits

38
questions deep

2,750
leaves

6.5
accounts per leaf

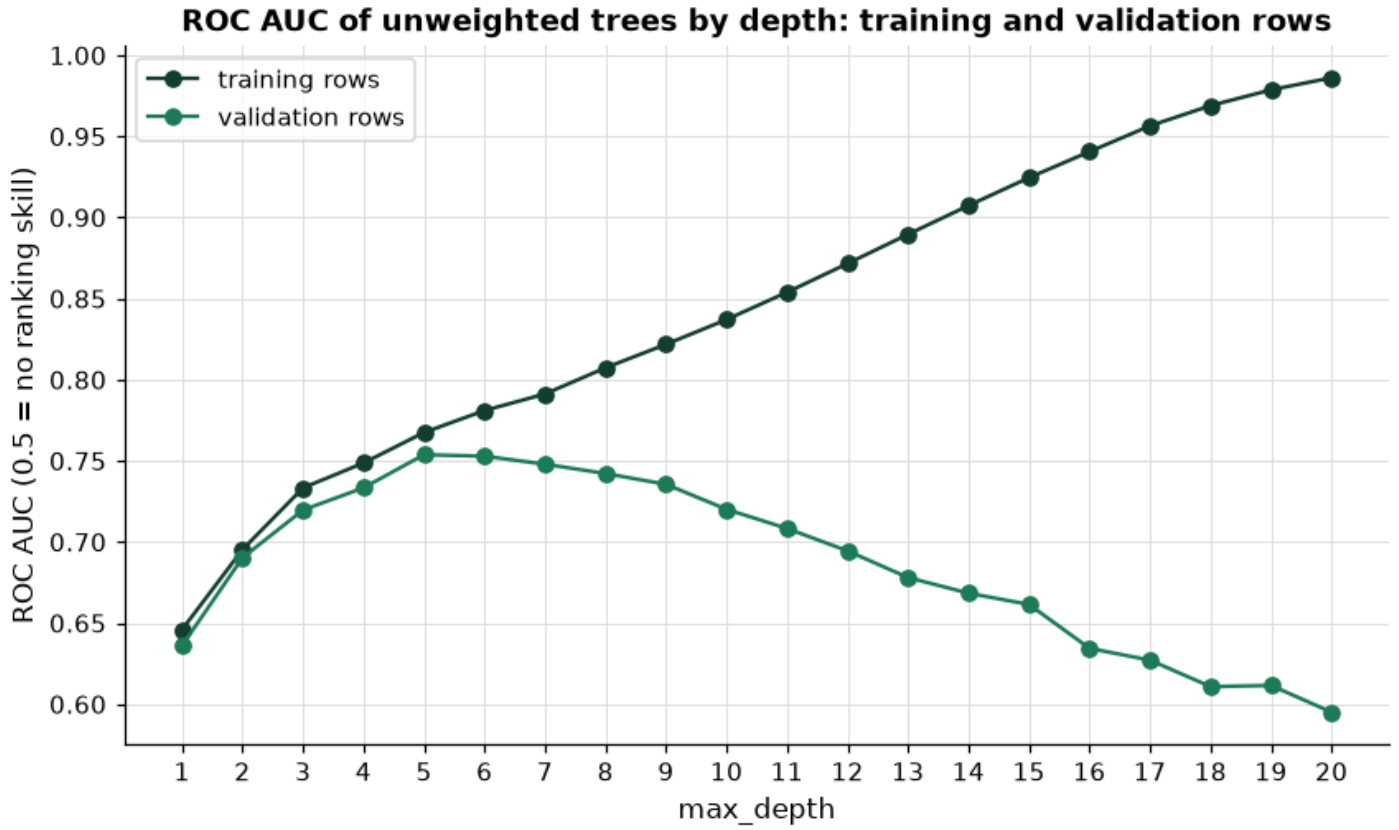
	ROC AUC	Recall	Precision
Training rows	0.9997	0.9709	0.9895
Validation rows	0.6161	0.4032	0.3888



Training recall is not 1: 48 groups of identical accounts (413 in all) hold both outcomes.

Nearly memorised on training rows, much worse on rows it never saw: overfitting.

Overfitting grows with depth



depth 5

validation peak

0.7538

validation ROC AUC there

0.9862

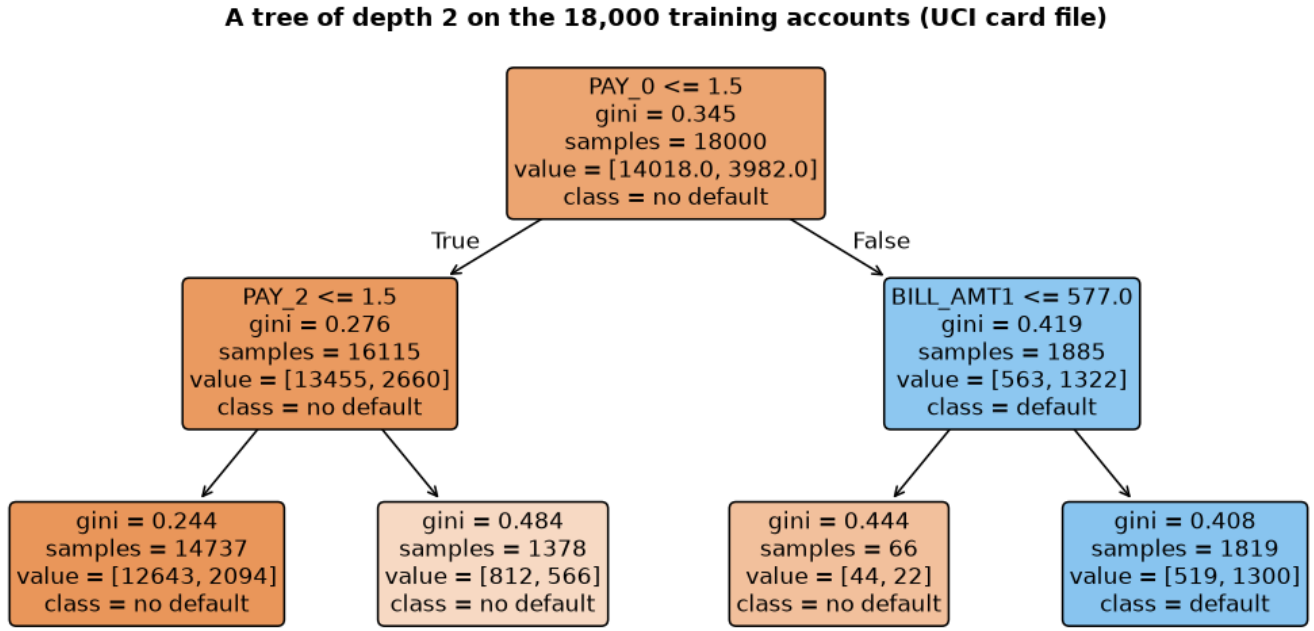
training, depth 20

0.5950

validation, depth 20

The space between the lines is overfitting. The default tree is at depth 38.

Reading a tree: depth 2



In this file's training rows, of accounts with status 2 or more and a bill above 577, 71.5 percent defaulted. A description, not a rule.

How class weights work

The problem	An unweighted tree flags a leaf only where defaults are the majority: few accounts when defaults are rare
The weights	balanced: each default counts more, each non-default less, so the classes weigh the same in total
In the tree	Weighted rows score the splits and set each leaf's share, so leaves lean to the rare class
Not resampling	No row is added or dropped
What comes out	A tree that flags more accounts; its output is a weighted share, a score and not a probability

Class weights: make the rare class count

2.2602

weight of a default

0.6420

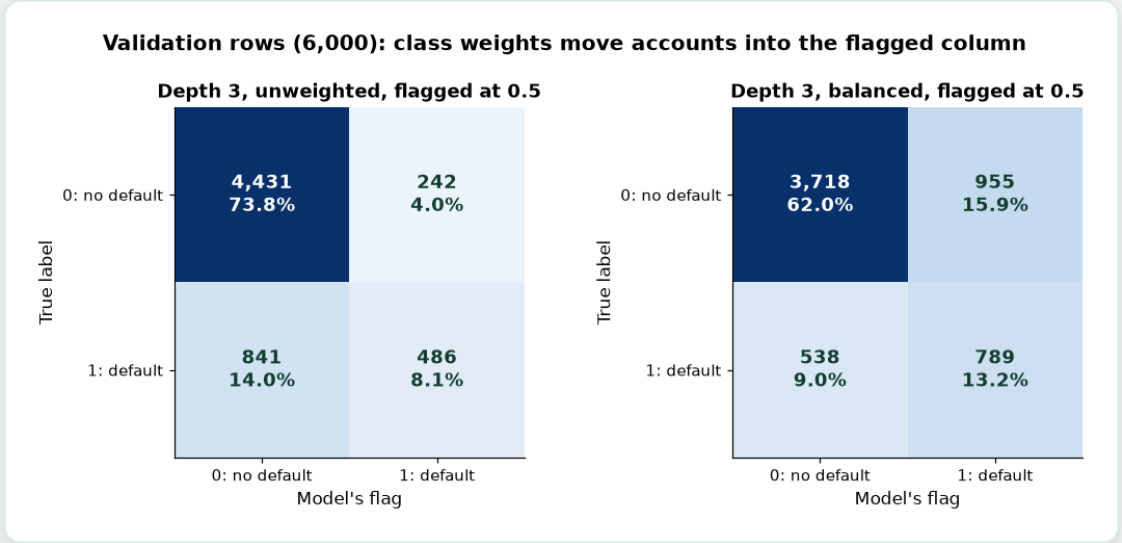
weight of a non-default

9,000

total weight of each class

Depth 3, validation	ROC AUC	Recall	Precision
Unweighted	0.7197	0.3662	0.6676
Balanced	0.7491	0.5946	0.4524

Weighted output is a score: 0.2770 in a leaf of 8,346 accounts where 0.0981 defaulted.



How pre-pruning works

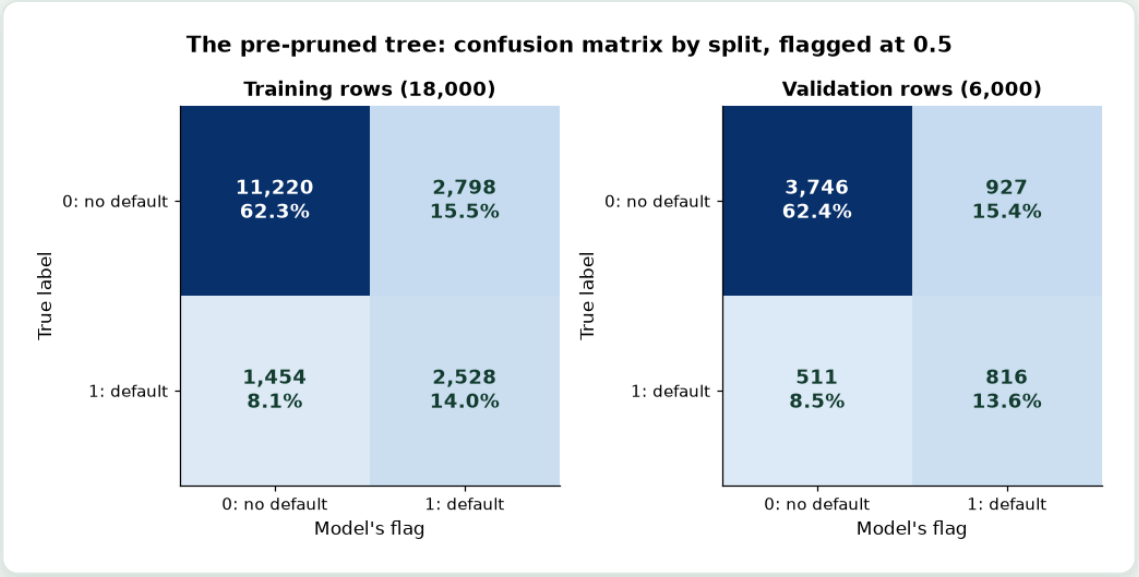
The problem	An unlimited tree splits until every leaf is pure, which fits the noise
max_depth	Refuses any question beyond that many levels
min_samples_leaf	Refuses any split that would leave a leaf with fewer training accounts than this
Which values	Cross-validation inside the training rows: five folds, best average score; validation stays out
What comes out	A smaller tree, a score for each pair of settings, and the pair that won

Pre-pruning: stop the tree early

Setting	Stops
`max_depth`	a path longer than this
`min_samples_leaf`	a leaf smaller than this

```
cv=mlkit.cv_folds()   # StratifiedKFold(5,
                      #   shuffle=True, 42)
```

Chosen by five-fold cross-validation inside the 18,000 training rows. 38 tests pass.



depth 8
leaves of at least 200

0.7664
cross-validated ROC AUC

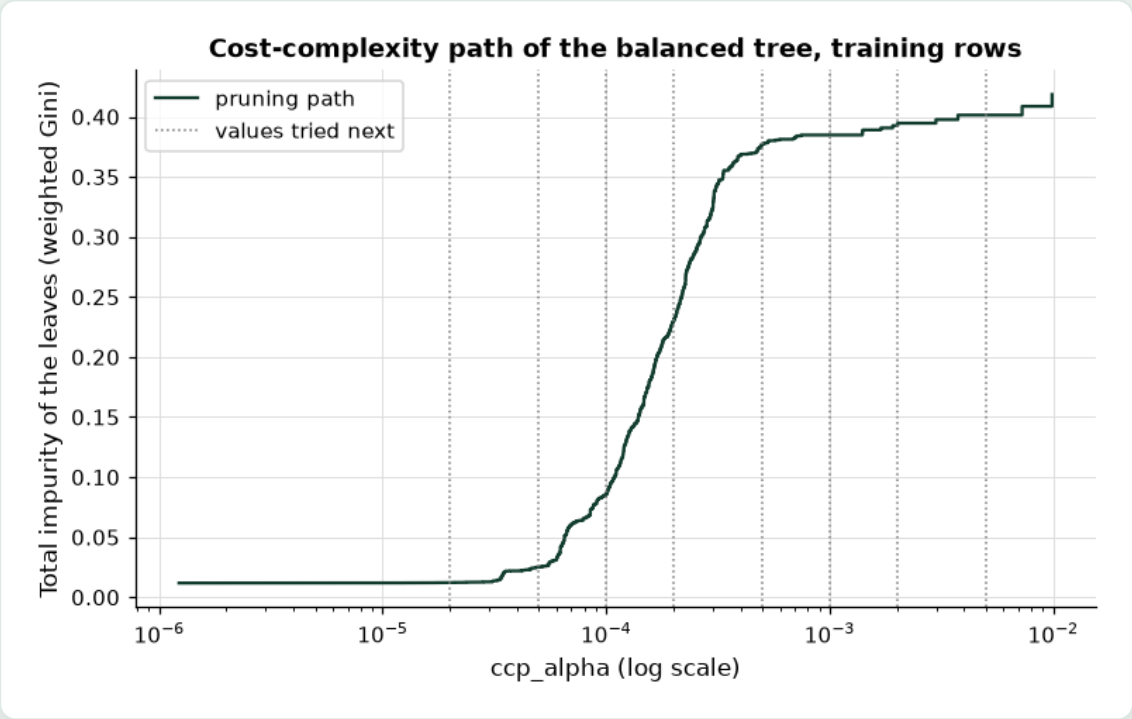
52
leaves

0.7633
validation ROC AUC

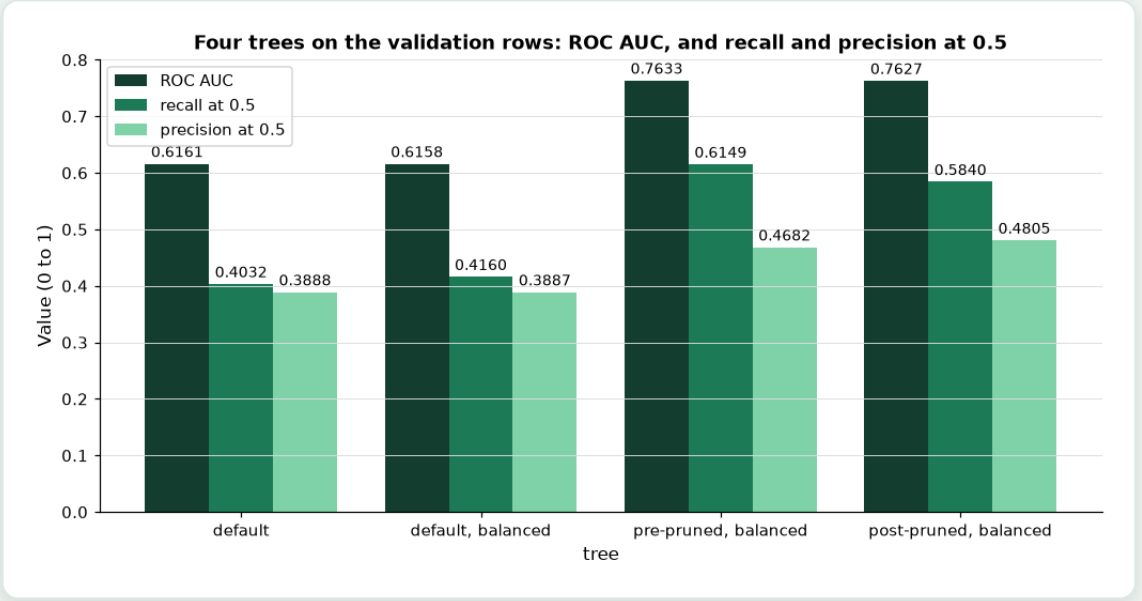
How post-pruning works

Grow first	Grow the full tree, then cut back the splits that buy the least
The score	Total impurity of the leaves plus alpha times the number of leaves; the lowest wins
Alpha	At 0 nothing is cut; as it rises the weakest splits go first, until only the root is left
The path	Every alpha at which the tree loses a split
What comes out	Nested trees from the full one to the root, and the one pruned at the chosen ccp_alpha

Post-pruning: grow, then cut back



1,743 alphas; best ccp_alpha 0.001 (cross-validated 0.7598). Post-pruned: 12 leaves, validation ROC AUC 0.7627.

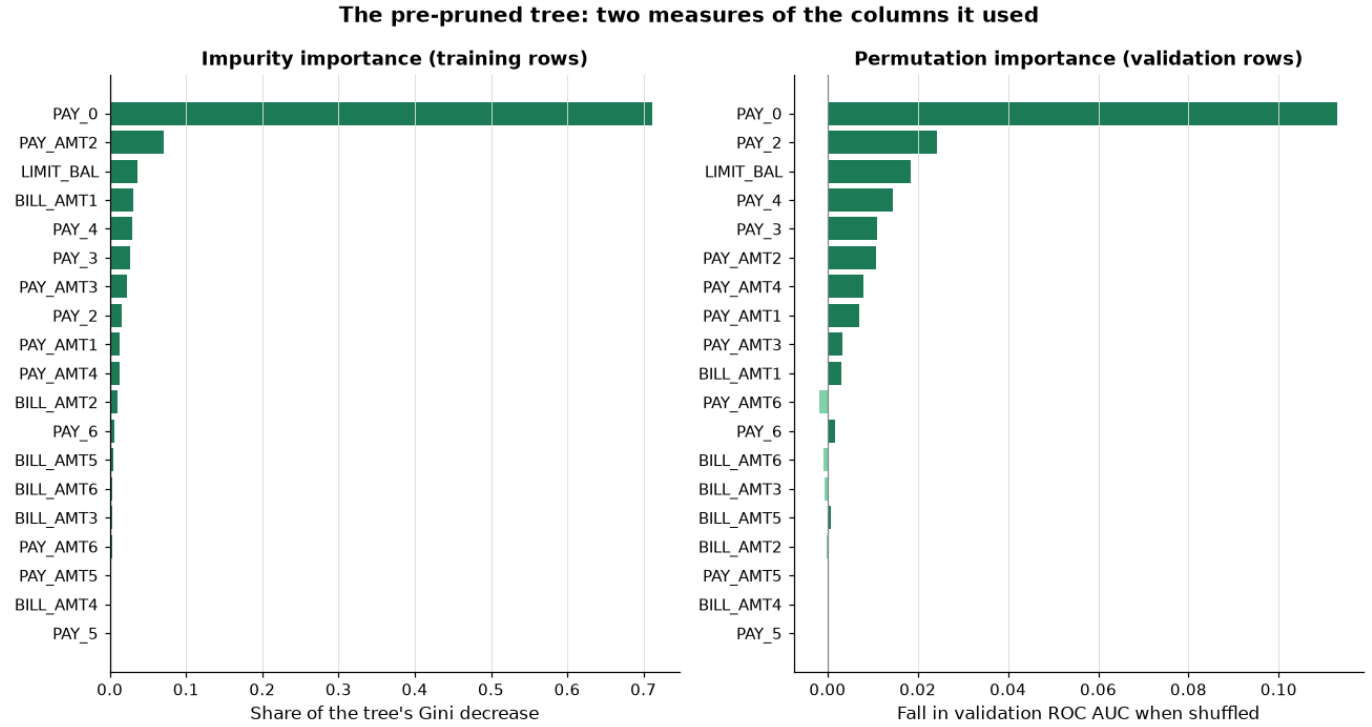


Balanced trees ask PAY_0 <= 0.5 first: weights change the questions.

How feature importance works

Impurity	The decrease in Gini from every split on a column, weighted by the accounts that reach it; sums to 1
Where	Computed on the training rows, while fitting
Permutation	Shuffle one column on the validation rows; the fall in ROC AUC is its importance
You control	The number of repeats and the seed
Not said	Neither measure says that a column causes defaults

Feature importance, and what it does not mean



Not cause

The tree used it to sort accounts

Not direction

Higher value: more or fewer?

Shared

Related PAY_ columns mask each other

Training rows

Impurity favours many split points

Impurity: training rows. Permutation: fall in validation ROC AUC when a column is shuffled.

Scaling does not change a tree

A split asks above or below; scaling keeps the order of values and thresholds.

6,000

validation accounts, same output (pre-pruned tree)

5

accounts exactly on a threshold (default tree)

Same tree, node for node. A tree needs no scaling.

Every setting of DecisionTreeClassifier

Tree shape and size

max_depth, min_samples_leaf, ccp_alpha: tune first. Also max_leaf_nodes, min_samples_split, min_impurity_decrease, min_weight_fraction_leaf.

How a split is chosen

criterion, splitter, max_features: rarely matter for one tree. max_features returns in Course 5.

Randomness and class weights

random_state: always set. class_weight: a choice about which mistake to avoid, section 9.5.

Constraints

monotonic_cst: not used in the course.

13

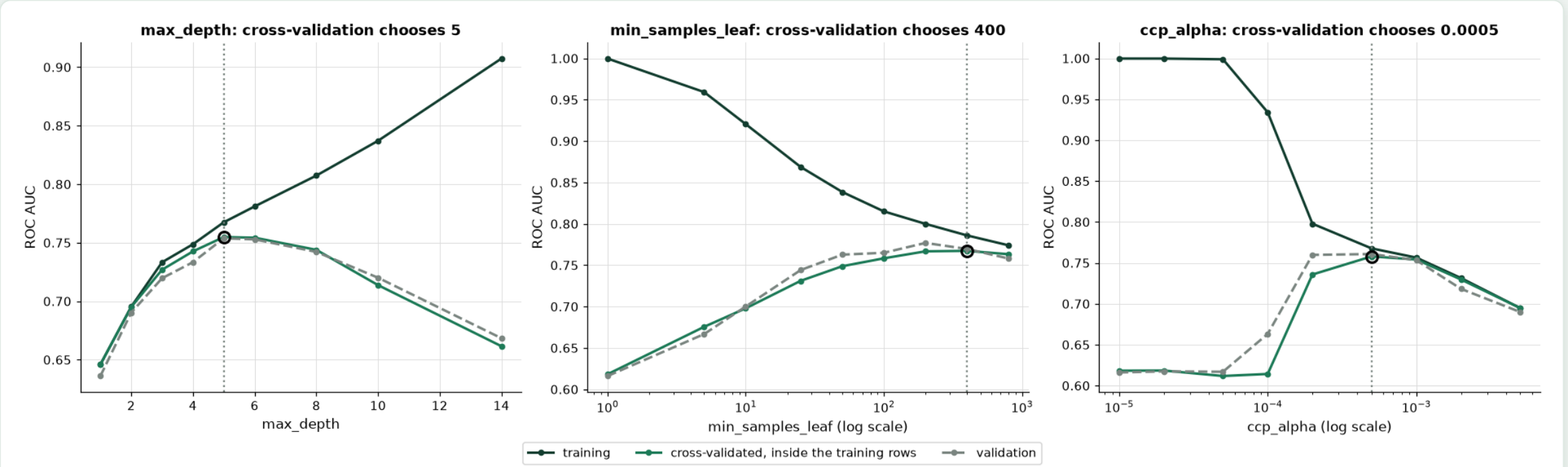
settings in the library

3

to tune first

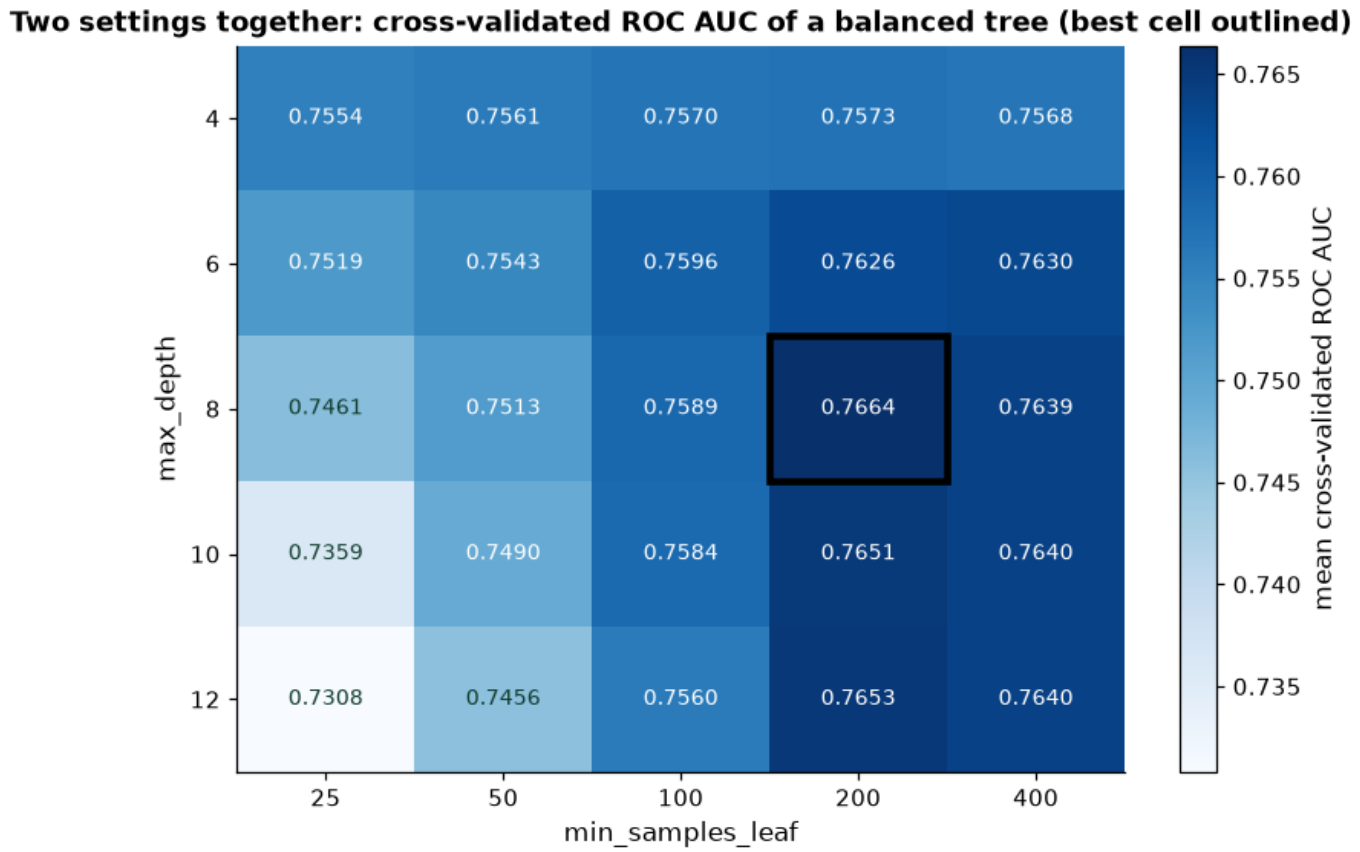
The regressor has the same settings, minus class_weight, and criterion defaults to squared_error.

Sweeping the three settings to tune first



Unweighted trees, one setting at a time. Chosen by cross-validation: max_depth 5, min_samples_leaf 400, ccp_alpha 0.0005. Pick the top of the cross-validated line, never the training line.

Two settings together



8

best depth

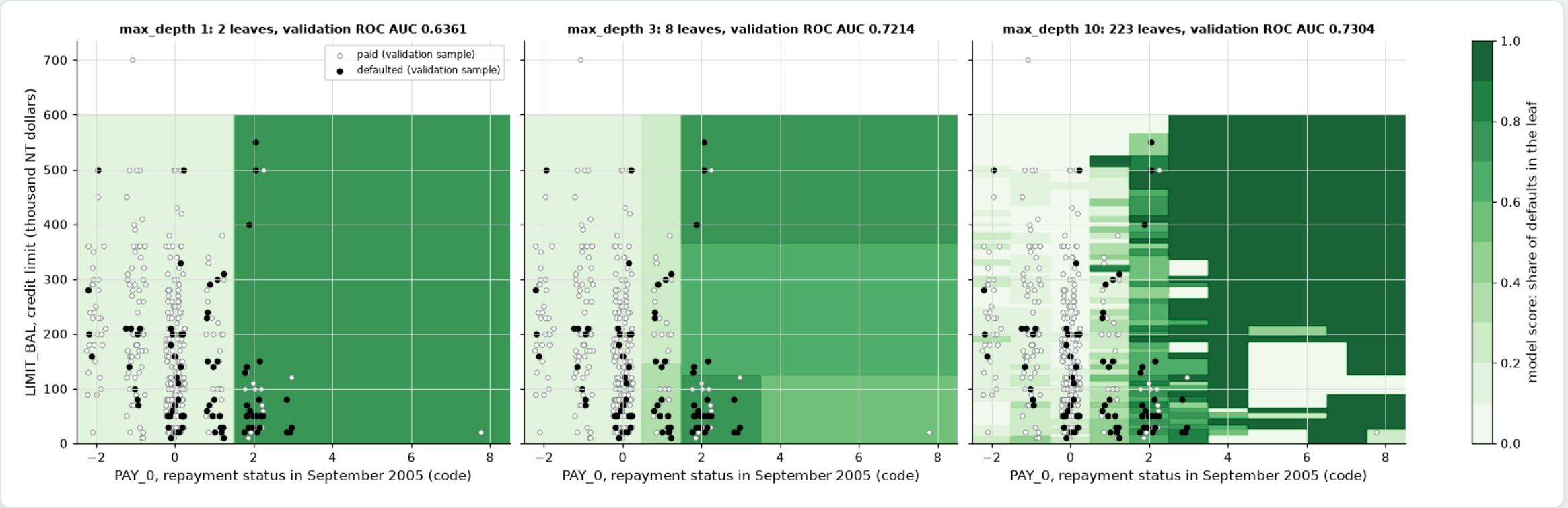
200

smallest leaf

Down a column depth helps, then stops. Along a row bigger leaves help a deep tree most. The worst cells: deep trees with the smallest leaves.

The settings interact, so a search tries them together.

What the settings do to a tree's shape



depth 1: 2 leaves, AUC 0.6361; depth 3: 8 leaves, AUC 0.7214; depth 10: 223 leaves, AUC 0.7304. Every cut is horizontal or vertical.

Excel side by side: a tree by hand

Excel side by side (tested in Excel)

`=COUNTIFS(A2:A18001,"<=1.5")`	16,115 accounts on the left
`=COUNTIFS(A2:A18001,"<=1.5",D2:D18001,1)`	2,660 defaults on the left
`=-(A1/(A1+B1)*LOG(A1/(A1+B1),2)+B1/(A1+B1)*LOG(B1/(A1+B1),2))`	entropy of the 6 / 4 box: 0.9710 bits
`=1-(G2/G1)^2-((G1-G2)/G1)^2`	Gini 0.3446
`=G3/G1*G8+G5/G1*G9  =G7-G10`	0.2906, decrease 0.0539
`=AVERAGEIFS(D2:D18001,A2:A18001,">1.5",C2:C18001,">577")`	leaf share 0.7147
`=IF(A2<=1.5,IF(B2<=1.5,0,0),IF(C2<=577,0,1))`	the tree's flag, all rows
`=H1/(H1+H2)  =H1/(H1+H3)`	recall 0.3052, precision 0.6935

Excel has no decision tree. It can count, weigh, and apply a tree Python fitted.

Save the day: tag the pruned trees

1. Code

mlkit.py and test_mlkit.py

2. Unpruned

results.csv: two trees

3. Pruned

two more rows, then git tag tree-pruned

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
default,validation,0.6161,0.2917,0.5000,0.7278,0.3888,0.4032,0.3959
"default, balanced",validation,0.6158,0.2926,0.5000,0.7262,0.3887,0.4160,0.4019
"pre-pruned, balanced",validation,0.7633,0.5248,0.5000,0.7603,0.4682,0.6149,0.5316
"post-pruned, balanced",validation,0.7627,0.4954,0.5000,0.7683,0.4805,0.5840,0.5272
```

A tag names one commit for good: git show tree-pruned:results.csv.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the search is fitted on training and validation rows (24,000), then scored on validation: 0.7645.

Code that runs is not the same as code that is right.

What you can do now

- A tree picks the question with the largest Gini decrease
- Left to grow it memorises: 0.9997 training, 0.6161 validation
- Class weights make defaults count; the output becomes a score
- Prune before or after, chosen by cross-validation on training rows
- Importance says what the tree used, never why

Week 2, Day 5 Case study: the card file, a threshold for recall 0.70, test rows opened once